

CYBERSECURITY LAB REPORT

SQL Injection Vulnerability in WHERE Clause

Allowing Retrieval of Hidden Data

Student Name	DINESH S.
Email	s90960665@gmail.com
Lab Level	Apprentice
Vulnerability	SQL Injection (SQLi)
Tool Used	Burp Suite
Completion Time	Approximately 2 minutes
Status	Successfully Completed

Practical SQL Injection Testing Report

Prepared for cybersecurity learning and lab documentation

1. Executive Summary

This report documents the successful completion of an Apprentice-level SQL Injection lab. The application contained an injectable product category filter. By intercepting the request with Burp Suite and modifying the category parameter, the SQL filtering logic was bypassed and unreleased products were displayed. The lab was completed in approximately two minutes.

2. Objective

The objective was to identify and exploit a SQL injection vulnerability in the product category filter so that the application would return one or more products that were not normally released/displayed.

3. Vulnerability Overview

The vulnerable application uses user-controlled category input in a SQL WHERE clause. The intended query structure is similar to:

```
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

The weakness occurs because the category value can influence SQL syntax rather than being safely handled only as data.

4. Attack Flow

Step	Action	Result
1	Open lab and category filter	Normal category request observed
2	Intercept request with Burp Suite	Category parameter identified
3	Modify category parameter	SQL injection payload inserted
4	Forward modified request	Injected query is processed
5	Review response	Unreleased products become visible

5. Exploitation Procedure

Step 1 — Intercept the Request

The product category request was intercepted using Burp Suite. The request contained a category parameter similar to:

```
category=Gifts
```

Step 2 — Inject SQL Syntax

The category value was replaced with the lab's intended test payload:

```
'+0R+1=1--
```

Step 3 — Send the Modified Request

The modified request was forwarded to the application. The injected OR condition evaluates to true, while the SQL comment marker prevents the remaining original condition from restricting the results.

Conceptual Query

```
SELECT * FROM products WHERE category = '' OR 1=1 --' AND released = 1
```

6. Result and Verification

The response displayed one or more products that were not normally visible because they were unreleased. This confirmed that the SQL injection successfully altered the application's query logic.

Finding	SQL Injection
Affected Function	Product category filter
Parameter	category
Observed Impact	Unreleased products displayed
Completion	Successful
Time	Approximately 2 minutes

7. Security Impact

- Unauthorized retrieval of application data.
- Bypassing intended application filtering logic.
- Potential exposure of records that should remain hidden.
- In more severe real-world cases, SQL injection can potentially enable broader database compromise depending on permissions and query context.

8. Root Cause

The root cause is unsafe handling of user-controlled input in the SQL query. The application permits input to influence the SQL statement structure instead of safely binding it as a parameter.

9. Recommended Mitigations

- Use parameterized queries or prepared statements.

- Never concatenate untrusted input directly into SQL statements.
- Validate and constrain user input where appropriate.
- Use database accounts with the minimum required privileges.
- Avoid exposing detailed database errors to users.
- Perform regular secure code reviews and SQL injection testing.

10. Evidence / Screenshot Placeholders

Insert the following screenshots into the final submission if required:

- Original category request in Burp Suite.
- Modified request containing the SQL injection payload.
- Response showing the previously hidden/unreleased products.
- Lab completion/success confirmation.

11. Tools and Environment

Tool / Platform	Purpose
Burp Suite	HTTP request interception and modification
Web Browser	Accessing the vulnerable lab application
PortSwigger Web Security Academy	Authorized SQL injection training environment

12. Conclusion

The SQL injection vulnerability in the product category filter was successfully identified and exploited within the authorized training environment. The category parameter was manipulated using the lab payload, causing the application to return unreleased products. The successful completion in approximately two minutes demonstrates a clear understanding of basic SQL injection concepts, HTTP request manipulation, and Burp Suite workflow.

LAB COMPLETION RECORD
SQL Injection — WHERE Clause
Status: SUCCESSFULLY COMPLETED
Completion Time: ~2 Minutes
DINESH S.